

3 Common Open Source IP Compliance Failures and How to Avoid Them

3 Common Open Source IP Compliance Failures and How to Avoid Them

Created on 2017-02-02 23:07

Published on 2017-02-05 18:53

Companies or organizations that don't have a strong open source compliance program often suffer from errors and limitations in processes throughout the software development cycle that can lead to open source compliance failures.

In part 3 of this series, we covered some of the risks that a company can face from license failures, including an injunction that prevents a company from shipping a product; support or customer service headaches; significant re-engineering; and more.

This time, we'll cover three common intellectual property failures, how they're discovered, and how to avoid them. And in part 5, we'll discuss the most common open source license compliance failures and how to avoid them.

3 Common Intellectual Property Failures

IP problems most commonly involve mixing source code that is licensed under incompatible or conflicting licenses (e.g., proprietary, third-party, and/or open source). Such admixtures may result in companies being forced to release proprietary source code under an open source license, thus losing control of their (presumably) high-value intellectual property and diminishing their capability to differentiate in the marketplace.

<i>Problem Type</i>	<i>How to Avoid</i>
Inserting open source code into proprietary or 3rd party code Occurs during development process when developers copy/paste open source code (aka "snippets") into proprietary or 3rd party source code	Offer training to increase awareness of compliance issues, open source (OS) licenses, implications of including OS code in proprietary or 3rd party code Conduct regular code scans of all project source code for unexpected licenses or code snippets. Require approval to use OS software before committing it into product repository
Linking of open source into proprietary source code (or vice versa – specific to C/C++ source code) Occurs as a result of linking software components that have conflicting or incompatible licenses	Offer training on linkage scenarios based on company compliance policy Regularly run dependency tracking tool to verify all linkage relationships; flag any issues not in line with compliance policies
Inclusion of proprietary code into an open source component Occurs when developers copy/paste proprietary source code into OS software	Train the staff Conduct regular source code inspections Require approval to include proprietary source code in OS components

Problem #1: Inserting open source code into proprietary or third party code

This occurs during the development process when developers copy/paste open source code (aka "snippets") into proprietary or 3rd party source code.

- *How it's discovered:* By scanning the source code for possible matches with open source code.
- *How to avoid it:*
- Offer training to increase awareness of compliance issues, open source (OS) licenses, implications of including OS code in proprietary or 3rd party

code.

- Conduct regular code scans of all project source code for unexpected licenses or code snippets.
- Require approval to use OS software before committing it into a product repository.

Problem #2: Linking of open source into proprietary source code (or vice versa – specific to C/ C++ source code)

This occurs as a result of linking software components that have conflicting or incompatible licenses.

How it's discovered: With a dependency-tracking tool that allows discovery of linkages between different software components and identifies if the type of linkage is allowed per a company's OS policies.

How to avoid it:

- Offer training on linkage scenarios based on company compliance policy
- Regularly run a dependency tracking tool to verify all linkage relationships and flag any issues not in line with compliance policies.

Problem #3: Inclusion of proprietary code in an open source component

This happens when developers copy/paste proprietary source code into OS software.

How it's discovered: By scanning source code. A tool will ID source code that doesn't match what's provided by the OS component, triggering various flags for an audit.

How to avoid it:

- Train the staff
- Conduct regular source code inspections
- Require approval to include proprietary source code in OS components.

The following is adapted from "Open Source Compliance in the Enterprise" published by The Linux Foundation (free e-book download), a complete guide to creating compliance processes and policies for your organization.

Original post appeared on Linux.com.
